

Capturing Emerging Complexity in Lenia

Sanyam Jain
Dept of Computer Science
Østfold University College
Halden, Norway
sanyamj@hiof.no

Aarati Shrestha
Dept of Computer Science
Østfold University College
Halden, Norway
aaratis@hiof.no

Abstract—This research project investigates Lenia, an artificial life platform that simulates ecosystems of digital creatures. Lenia’s ecosystem consists of simple, artificial organisms that can move, consume, grow, and reproduce. The platform is important as a tool for studying artificial life and evolution, as it provides a scalable and flexible environment for creating a diverse range of organisms with varying abilities and behaviors. Measuring complexity in Lenia is a key aspect of the study, which identifies the metrics for measuring long-term complex emerging behavior of rules, with the aim of evolving better Lenia behaviors which are yet not discovered. The Genetic Algorithm uses neighborhoods or kernels as genotype while keeping the rest of the parameters of Lenia as fixed, for example growth function, to produce different behaviors respective to the population and then measures fitness value to decide the complexity of the resulting behavior. The grid world with best fitness is selected and given a chance to carry the genetic material forward in further generations. First, we use Variation over Time as a fitness function where higher variance between the frames are rewarded. Second, we use Auto-encoder based fitness where variation of the list of reconstruction loss for the frames is rewarded. Third, we perform combined fitness where higher variation of the pixel density of reconstructed frames is rewarded. All three experiments are tweaked with pixel alive threshold and frames used. Finally, after performing nine experiments of each fitness for 500 generations, we pick configurations from all experiments such that there is a scope of further evolution, and run it for 2500 generations. Results show that the kernel’s center of mass increases with a specific set of pixels and together with borders the kernel try to achieve a Gaussian distribution. We also compare our results with running evolution on known and handcrafted kernels. Overall, this project aims to deepen our understanding of artificial life and evolution by investigating the potential of Lenia as tools for modeling complex systems and emergent phenomena in open-ended Artificial General Intelligence.

Index Terms—Continuous CA, Lenia, Evolutionary Computation, Artificial Life

I. INTRODUCTION

Open-endedness is important in AI [1], [2] because it enables the development of more flexible, creative, and autonomous systems that can solve a wider range of problems and tasks. It also facilitates the emergence of unexpected and potentially useful behaviors or solutions that may not have been anticipated by human designers. In addition, open-endedness allows AI systems to continue to learn and improve over time, adapting to changing environments and evolving to meet new challenges. Lenia is a digital life form created by mathematician and computer scientist Bert Wang-Chak Chan in 2019 as Independent researcher [3], [4]. It is a fascinating

and beautiful example of how simple rules can give rise to complex behavior in a digital system. It is a type of artificial life simulation that uses a continuous cellular automaton to generate complex, evolving patterns. Lenia differs from traditional cellular automata in that its cells are not limited to discrete states like “on” or “off”. Instead, each cell can take on a continuous range of values, which allows for more fluid and organic patterns to emerge. Lenia is also unique in that it allows for a wide range of parameters to be adjusted, such as the size and shape of the grid, the rules for how cells interact (kernel), and the speed at which the simulation runs (growth function). This flexibility has led to a diverse array of visual and auditory creations, ranging from mesmerizing abstract patterns in Sensorimotor Lenia to complex rhythmic compositions in particle Lenia.

One of the reasons why most of the research in Lenia is focused on finding new species is because Lenia is an open-ended, evolving system that has the potential to generate an infinite number of unique and complex patterns. This makes it an ideal system for exploring the principles of emergent behavior and evolutionary dynamics. By searching for new species in Lenia, researchers can uncover new and unexpected patterns that may have applications in fields such as digital art, music, and data visualization. These patterns can also provide insights into the underlying principles that govern complex systems and help us better understand the behavior of biological systems. In addition, the search for new species in Lenia can also lead to the discovery of novel algorithms and computational techniques that can be applied to other fields. For example, the methods used to analyze and classify Lenia patterns can be applied to biotechnology, genetic evolution of real organisms and synthetic biology. However in our research, we keep the system open-ended, self organising and let evolution produce behaviours in the Lenia subsystem.

Cellular automata (CA) have proven applications in the field of self-organized control. In a paper [5] authors describe how they used a variant of CA called Neural Cellular Automata (NCA) to develop a control system for a cart-pole agent. The goal of the control system is to keep the pole balanced while moving the cart in response to external disturbances. This work used neural cellular automata to control a cart-pole agent through deep Q-learning, demonstrating stable behavior over many iterations and exhibiting life-like phenomena such as regeneration and robustness to disruptions. Further, highly

complex and self organising versions of classical Lenia such as Sensorimotor Lenia [6], Flow Lenia [7], and Energy based Particle Lenia [8] shows advanced physics, chemistry and biology of the matter.

For the rest of the paper, we provide Related Work and relevant references in section 2. Further section 3 proposes methodology, followed by section 4 which provides detailed explanation about Experimental setup. In section 5, we provide experimental results and required discussion from the interpretation of the results. Finally, we conclude our research paper with future scope and improvements.

II. RELATED WORK

A. Lenia

Lenia [3], [4] is a digital life simulation software that utilizes a unique approach to simulate various patterns and behaviors through the use of an abstract concept called the "growth function." The growth function in Lenia represents the process of growth and decay of various shapes and structures, which is governed by two important parameters: μ and σ .

The μ parameter in Lenia's growth function is the central value or the mean of the function, which determines the overall behavior of the shapes or structures in the simulation. The σ parameter, on the other hand, represents the standard deviation or the spread of the function. It controls the degree of randomness and unpredictability in the growth and decay process.

To understand the concept of kernel ring shell in Lenia, we need to first define the "kernel" of the growth function. The kernel in Lenia refers to the central shape or structure around which the growth function operates. The kernel can be any shape or structure, such as a point, a line, a circle, a square, etc. The kernel is surrounded by a series of concentric rings called the "kernel ring shell." The kernel ring shell consists of a series of smaller shapes or structures that surround the kernel, and the growth function operates on each of these shapes to produce complex patterns and behaviors.

The μ and σ parameters of the growth function are used to control the growth and decay process of each shape within the kernel ring shell. The μ value determines the central behavior of each shape, while the σ value controls the degree of randomness and unpredictability in the growth and decay process. Together, the μ and σ parameters of the growth function enable Lenia to generate a wide range of complex patterns and behaviors, ranging from simple geometric shapes to intricate fractals and cellular automata.

In the Lenia system, altering the values of these parameters for the growth and kernel functions can generate different species (we describe growth function and kernel in next paragraph). Over 500 species have been discovered so far [12], with each residing in a unique combination of these parameters. The values can be varied through evolutionary computation, but the outcome is often a dead or chaotic grid. Essentially, Researchers typically use smooth ring kernel shells with different values of μ and σ . To conform to the taxonomy, the discovery of new species must pass specific

benchmarks. Lenia uses a kernel, a circular matrix that acts as a filter to smooth the state of the simulation board. The kernel is circular, with a diameter that is equal to the size of the kernel and centered around the current cell being processed. During convolution, the kernel is slid over the entire board, and for each cell, the corresponding values in the kernel and surrounding cells are multiplied and summed to generate a new value for that cell. Lenia is a highly complex system that allows researchers to manipulate the μ and σ values of the growth and kernel functions to create a diverse array of species. The discovery of new species is an active area of research, with efforts focused on using interactive evolutionary systems. The Gaussian function is used to calculate the values in the kernel, and it has many applications in various fields, including image processing. Gaussian filtering is a common technique used to smooth or blur images by convolving the image with a Gaussian kernel, with the size and shape of the kernel depending on the desired effect.

$$G(U) = \frac{1}{2\pi\sigma^2} e^{-\frac{(U-\mu)^2}{2\sigma^2}} \quad (1)$$

where σ is the standard deviation of the Gaussian function, μ is its mean, and U is the convolved weighted sum of the kernel and board output. The Gaussian kernel is used for smoothing and blurring the simulation board, and it is a circular matrix that is centered around the current cell. The number of smooth rings or "shells" in the Gaussian kernel is determined by the "peaks" parameter, which specifies the amplitude of the peaks for each shell. The diameter of the kernel is specified by the user, and the parameter "R" is calculated as half of the diameter to generate a square grid of pixels around the current cell. The distance from the center of the kernel to each shell for every pixel is calculated using the Euclidean norm and stored in the "D" variable. The peak value for each pixel is determined by multiplying the peak value of the closest shell with the normalized distance from the center of the kernel to the closest shell for that pixel, and the Gaussian function is applied to each pixel using specified values of μ and σ . The resulting smoothed kernel can be used to update the state of the simulation board. The growth function updates the simulation board state based on the neighborhood sum, which is calculated as the convolved value of the kernel over board. The Gaussian growth function is then applied to the neighborhood sum with two parameters: the mean or center of the curve m and the standard deviation or spread of the curve. This function provides a smooth and continuous function that gradually changes the board state in a more fluid-like manner. The smoothness of the Gaussian function prevents sudden and unrealistic patterns. By adjusting the mean and standard deviation parameters of the Gaussian function, each cell's growth or decay can be precisely controlled. This allows for a wide range of patterns, from simple geometric shapes to complex and intricate structures in the Lenia simulation. A detailed view of kernel and growth function can be seen in Figure 1

A precise Lenia algorithm works as shown in Algorithm 1.

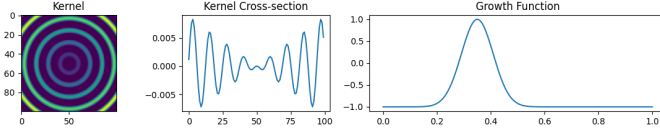


Fig. 1. Visualising Kernel, Gaussian Kernel Cross-Section Function and Gaussian Growth Function

- 1) Take a 2D array (world A) of real values between 0 and 1, initialized with an initial pattern A_0 .
- 2) Calculate weighted sums of A with a predefined array (kernel K), which is equivalent to calculate the convolution $K * A$; the kernel K has radius R , forming a ring or multiple concentric rings (parameter β = list of peak value of each ring).
- 3) Apply a growth mapping function G to the weighted sums; the growth mapping G is any unimodal function (parameters μ = growth center, σ = growth width).
- 4) Add a small portion dt of the values back to the array A .
- 5) Finally clip the states of A to between 0 and 1.
- 6) Repeat steps 2-5 for each time-step.

$$A_{t+dt} = [A_t + dt.G(K * A_t)]_{0,1} \quad (2)$$

In this formula, A_t and A_{t+dt} represent the values of the array world A before and after the update at time t and $t + dt$, respectively. The operator $*$ represents the convolution operation between the kernel K and the array A_t . The function $G(\cdot)$ represents the growth mapping function with parameters μ and σ , applied to the weighted sums of the convolution result. $0,1$ represents the range of valid values for the array in equation 2.

B. Evolutionary Cellular Automata

Evolutionary cellular automata (CA) [9] can be used to evolve Lenia, a digital life form with complex behaviors and morphology. The search for useful CA rules is challenging due to the difficulty of predicting the outcome of a given rule set from a starting configuration. To overcome this challenge, an evolutionary search is a natural choice for exploring CA with desired properties. However, traditional CA are difficult to evolve naively, and variants of the framework are often used to improve evolvability. For example, representations inspired by genetic programming have been used for the transition function. While traditional CA have a consistent transition function for all cells, Lenia experiments have explored the use of homogeneous growth functions while mutating the kernel using genetic algorithm. The ability to evolve CA rules allows for the exploration of new and interesting behaviors and structures is same in Lenia considering rules analog to kernel and growth functions, leading to a better understanding of its complex and diverse universe.

III. PROPOSED METHODOLOGY

We propose modified versions of two known approaches called compression based metric and deviation based approach

to evaluate cellular automata (CA) [11]. Previously these approaches have been studied on classical CA, we propose novel modified approaches namely, AutoEncoder (AE) based compression metric, Variation over Time (VoT) based deviation metric and a completely novel technique called as Auto Encoder Variation Over Time (AEVoT) which is built on top of known AE and VoT approaches. We describe each of them as follows:

A. Variation Over Time

Measuring complexity in Lenia can be a challenging task, as the board can exhibit a wide range of dynamic behaviors over time, ranging from simple repetitive patterns to highly complex and unpredictable behaviors. One way to quantify complexity is to measure the variation in the state of the board over time. One approach is to measure the number of active cells on the board at each time step. The number of active cells can provide a rough estimate of the complexity of the system, as more active cells typically indicate a higher degree of complexity. However, it is important to note that the number of active cells can vary significantly between different time steps, and may not be a reliable measure of complexity in all cases. A threshold (a hyper-parameter) is used to determine the alive state of the grid. For pixel values smaller than threshold, cells are dead and conversely for alive cells. Count from each grid is stored in an array and population standard deviation (pstd) is calculated on that list which collects the alive cells count of each frame. It is the deviation based measure of complexity, which calculates the differences of the distribution of active cells on the board at certain time steps. Deviation is a measure of the randomness or differences in states of a system, and can be used to quantify the variation of complexity in a system over time. The VoT approach is particularly useful in identifying temporal patterns and trends in the system's behavior, which can be indicative of its underlying complexity. One advantage of this approach is that it does not require prior knowledge of Lenia simulation as AE needs. A minor disadvantage is that if the simulation is heading towards periodic doubling of chaos, it will still provide a good fitness for such behavior. An overview of the approach is shown in Figure 2

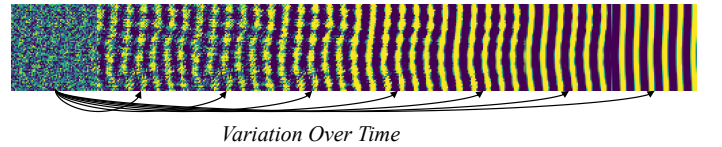


Fig. 2. Variation over time approach to measure complexity

B. Compression based

Another approach, uses encoder-decoder network to compress the input images. Encoder-decoder networks have been proposed as a way to measure the complexity of cellular automata such as Lenia. In this approach, the encoder network is trained to encode the current state of the Lenia board into

a lower-dimensional feature space, while the decoder network is trained to decode the features back to the original state. The key idea is that the amount of information that is lost during the encoding and decoding process can be used as a measure of the complexity of the Lenia board. The calculated Reconstruction loss (using a distance measure) will then decide whether the input is complex or ordered. Hypothesis is, if the input is complex and random, it is challenging for decoder to reconstruct the input and hence it will result higher reconstruction loss while in the ordered behaviour the reconstruction loss would be small. The input frames are reconstructed using the trained AE and Mean Squared Error (MSE) loss is stored in a list. The MSE is calculated between original frame and reconstructed frames. After that, population standard deviation (pstdd) is calculated between those errors. If the list of errors has a higher pstdd then it is hypothesised that there is a larger variance between the frames generated using the evolved kernel. However, if there is a smaller change in errors in list, it means that the behaviour shown by the evolved kernel in Lenia is predictable or too orderly due to which the errors are similar throughout the frames. To train the encoder-decoder network, a dataset of Lenia board states is first generated by running the simulation for a certain number of steps. The encoder network is then trained to map each board state to a set of features that capture the key patterns and structures in the board. The decoder network is trained to reconstruct the board state from the features, with the goal of minimizing the reconstruction error. Advantage of using AE is performance. AE are better at representing random and ordered behaviors with a quantity of MSE loss. However a major disadvantage is that it needs to be trained on a meaningful dataset before plugging the bottleneck as a fitness function that performs reconstruction and calculates the error. An overview of the approach is shown in Figure 3

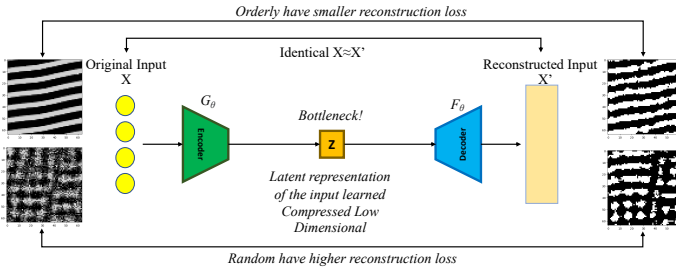


Fig. 3. Auto-encoder approach to measure complexity

C. Auto Encoder Variation Over Time

The Auto-Encoder based Variation over Time (AEVoT) approach combines the two previously described methods, AE and VoT, to measure the complexity of the Lenia patterns over time on reconstructed frames. It is very similar to the AE and VoT approaches, with a difference that VoT is calculated on reconstructed images from the AE. To use an AE for time series data, we can treat each time step as a separate input and train the AE to reconstruct the original sequence of

data. The reconstruction loss, which is the difference between the input and the reconstructed output, can then be used to measure the complexity of the time series data. By analyzing the variation in the reconstruction loss over time, we can gain insights into the complexity of the time series. For example, if the reconstruction loss is relatively constant over time, it may indicate that the time series is relatively simple or predictable. However, if the reconstruction loss varies widely over time, it may indicate that the time series is more complex and difficult to predict.

All three techniques discussed in this section work as fitness function in the standard genetic algorithm procedure. More details about its implementation is described in next section.

IV. EXPERIMENTAL SETUP AND RESULTS

The experimental nature of this research is quite significant. It explores wide range of concepts, combinations of different techniques which are applied on measuring complexity in grid-world to evolve emerging patterns and behaviors using Evolutionary Computation (EC) based Genetic Algorithm (GA). First, we outline the standard working of Genetic Algorithm used in the project. Second, we provide section-wise decomposition about each subsystem of the genetic algorithm for our use case. After that, we describe Lenia simulation which gets called by GA. Finally, we combine all approaches to list down the experiments that are performed in this project.

A standard Genetic Algorithm comprises of Initialising population, evaluating fitness, performing selection, reproduction using crossover, and mutation. This loop gets executed up to desired number of generations. Essentially, GA also has several parameters that tunes the performance of the GA. An overview of a Genetic Algorithm is shown in Figure 4.

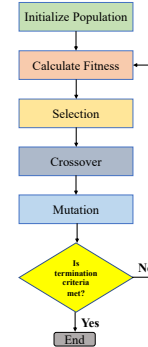


Fig. 4. Genetic Algorithm working pipeline

A high level overview of experimental setup is provided in Figure 5.

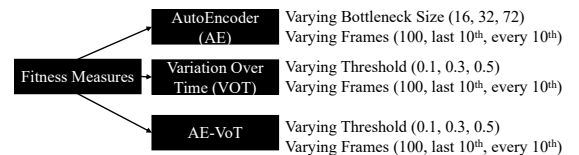


Fig. 5. Overview of experiments performed

A. Population

Initialised population comprises a random board and a random kernel. The initial *population_size* is defined by the variable *population_size*. The initial population consists of *population_size* number of individuals, each of which is an instance of the Individual class. The Individual class initializes each individual with a randomly generated kernel (using the *random_kernel_generator* function). The Population class is then used to create a population of *population_size* individuals with a random board using the *random_board_generator* function.

B. Evaluation

Three fitness measures proposed in this research are used for evaluating the generated behaviour in Lenia. Implementation setup for each of the fitness used are described as follows:

The *calc_fitness* function is a method of the Individual class which calculates the fitness value of an individual based on their genes and a given board. The *calc_fitness* function first creates an instance of the LeniaForGA class, passing in the individual's genes and the given board to create a simulation of the Lenia cellular automaton. The *run_simulation* method of the LeniaForGA class is called to run the simulation and store the generated board frames in a data dictionary and corresponding images in a directory. Further, this data dictionary is processed in three different ways namely "all produced frames for t time steps", "every 10th frame" and "last 10 frames" by three different techniques as follows:

1) *AE*: In this method, the input data is reconstructed to its original form and then the Mean Squared Error (MSE) between the original data and the reconstructed data is calculated. This is done for all the frames in the input data, and the MSE values are collected in a list. Finally, a standard deviation is calculated over the list of MSE values, which gives an overall measure of how well the input data was reconstructed. If the population standard deviation (pstd) of the list of MSE values is higher, it means that the fitness scores of the individuals in the population are more varied. This indicates that the individuals are performing very differently in terms of their ability to reconstruct the original input. On the other hand, if the pstd is lower, it means that the fitness scores are more similar across the individuals in the population. This indicates that the individuals are performing more similarly in terms of their ability to reconstruct the original input. Figure 6 shows Algorithm for AE method.

2) *VoT*: This method does not require any bottleneck to calculate fitness. It directly takes board input frames and uses pixel values to calculate pstd over frames. First, it uses a threshold to calculate the number of alive cells in each frame. For a continuous CA, the pixel values are in fraction and hence threshold also need to be a decimal value between 0 and 1. Further, after setting the threshold, each frame results a number of alive cells. All frames are processed and a list of number of alive cells is created. Finally, a pstd is calculated over that list. A lower standard deviation means that the CA is more stable and predictable, while a higher standard deviation indicates

Algorithm 1. Fitness Calculation

Input : Population of individuals with parameters
Output: Fitness values and population standard deviation (pstd)

```

1 Begin
2   Calculate fitness of each individual in the population using the
   following steps:
3   For each individual  $i$  in the population do
4     Reconstruct the input data  $x$  using the individual's parameters:
        $x_i = f(i)$ 
5     Calculate the Mean Squared Error (MSE) between the
       reconstructed data  $x_i$  and the original data  $x$ :
        $MSE_i = \frac{1}{n} \sum_{j=1}^n (x_{i,j} - x_j)^2$ 
6     Store the MSE value  $MSE_i$  in a list
7   Calculate the population standard deviation (pstd) of the list of
   MSE values:
8    $mean = \frac{1}{m} \sum_{i=1}^m MSE_i$ 
9    $variance = \frac{1}{m} \sum_{i=1}^m (MSE_i - mean)^2$ 
10   $pstd = \sqrt{variance}$ 
11 return Fitness values and the pstd value for the population

```

Fig. 6. Algorithm for AE method

more randomness or instability. Figure 7 shows Algorithm for VoT method.

Algorithm 2. Fitness Calculation using PSTD

Input : Board input frames
Output: Population standard deviation (pstd)

```

1 Begin
2   Calculate pstd over frames using the following steps;; Set a threshold
   value between 0 and 1 for the continuous CA; For each frame do
3     Calculate the number of alive cells in the frame by thresholding
       the pixel values; Store the number of alive cells in a list;
4   Calculate the population standard deviation (pstd) of the list;;
        $mean = \frac{1}{n} \sum_{i=1}^n num_i$ ;  $variance = \frac{1}{n} \sum_{i=1}^n (num_i - mean)^2$ ;
        $pstd = \sqrt{variance}$ ; return The pstd value;

```

Fig. 7. Algorithm for VoT method

3) *AEVoT*: This method combines both the approaches. First, reconstruction of the original frames is collected using AE. All the reconstructed frames are further passed to VoT method and produces a pstd over the list of alive cells count of each frame. And similar to VoT, a higher pstd is expected here as well. Figure 8 shows Algorithm for AEvOT.

C. Selection

The selection function used in the code is a Roulette Wheel Selection method, which selects individuals based on their fitness proportionate to the sum of the fitness of all individuals in the population. In this method, each individual is assigned a portion of the wheel proportional to its fitness, and a pointer on the wheel is set to a random starting position. The wheel is then spun, and the individual whose fitness region contains the pointer is selected for the next generation. The process is repeated until the desired number of individuals are selected. The function selects individuals for mutation, leaving out the elites which are simply copied to the next generation. The number of elites in the population is set to one for our case. Selection procedure for example, is provided in Figure 9

Algorithm 3. Auto-Encoder based Variation over Time (AEVoT)**Input** : Input frames of Lenia patterns**Output**: Population standard deviation (pstd) over list of alive cells count of each frame

```

1 Begin
2   Reconstruct the original frames using an auto-encoder (AE); For
   each input frame  $f$  do
3      $f_{AE} = AE(f)$ ; Calculate the number of alive cells in the
       reconstructed frame  $f_{AE}$  using a threshold;;
        $alive_f = count(p \geq threshold)$ ; Store the number of alive cells
        $alive_f$  in a list;
4   Calculate the population standard deviation (pstd) of the list of
       alive cells counts;;  $mean = \frac{1}{n} \sum i = 1^n alive_i$ ;
        $variance = \frac{1}{n} \sum i = 1^n (alive_i - mean)^2$ ;  $pstd = \sqrt{variance}$ ;
       return Population standard deviation (pstd) over list of alive cells
       count of each frame;

```

Fig. 8. Algorithm for AEVoT method

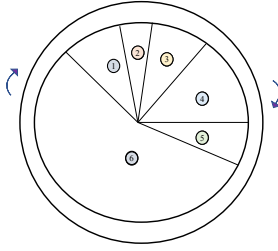


Fig. 9. Roulette Wheel Selection example

D. Mutation

It mutates an individual's genes by randomly altering each gene with a probability equal to the *mutation_rate* value. The *mutation_rate* value is set to 0.02, which means that each gene has a 2% chance of being mutated. If the random value generated for a gene is less than *mutation_rate*, the gene is randomly reassigned a value between 0 and 1, rounded to 3 decimal places. Given an individual's genes represented by a 2D *numpy* array *kernel*, the mutation function mutates each gene *kernel*[*i*][*j*] as follows:

- 1) If *random.random()* is less than *mutation_rate*, *mutatekernel*[*i*][*j*] by assigning it a random float value between 0 and 1, rounded to 3 decimal places.
- 2) Repeat the above step for all the genes in *kernel*.
- 3) Update the individual's genes with the mutated values.

Repeat the above steps for all the genes in *kernel*. Update the individual's genes with the mutated values. The mutation function can be seen as a probabilistic search operator that helps the GA to explore the search space efficiently. It introduces diversity into the population, which increases the chances of finding better solutions to the problem. However, if the mutation rate is too high, it can lead to premature convergence and prevent the GA from finding better solutions.

E. Evolutionary Lenia

The Lenia class is an implementation of a simulation model for 2D pattern growth, which can be modified easily with different growth functions and kernels. It is particularly useful

in the context of genetic algorithms for simulating the growth of organisms across multiple generations. The class has several attributes, including:

- 1) *sigma*: the standard deviation of the Gaussian function used in the growth function
- 2) *mu*: the mean of the Gaussian function used in the growth function
- 3) *dt*: the time step used in the simulation
- 4) *kernel*: a 2D *numpy* array representing the kernel used in the simulation
- 5) *frames*: the number of frames to generate in the animation
- 6) *frame_intervals*: the time interval between each frame in the animation
- 7) *lenia_board_state*: a dictionary used to store the state of the board at each generation, which is useful when running a genetic algorithm
- 8) *board*: a 2D *numpy* array representing the initial board state

The class also has several methods, including:

- 1) *growth_function1*: a method that takes a *numpy* array *U* as input and applies a Gaussian function with mean *mu* and standard deviation *sigma* to produce a growth rate between -1 and 1
- 2) *show_board*: a method that displays the board and returns the figure and *imshow* objects
- 3) *animate*: a method that generates the animation by calling *animate_step* for each frame
- 4) *animate_step*: a method that updates the board for a given frame and renders it
- 5) *save_animation*: a method that saves the animation in either .gif or .mp4 format
- 6) *normalise_kernel*: a method that normalizes the kernel by dividing it by the sum of its values
- 7) *plot_kernel_info*: a method that plots information about the kernel and growth function, and optionally saves it as a file
- 8) *run_simulation*: a method that runs the simulation and generates the animation

The *init* method initializes the class with the given *kernel* and *board*, and sets default values for other parameters. In summary, the Lenia class provides a flexible and extensible way to simulate and visualize the growth of 2D patterns, which can be easily integrated into a genetic algorithm framework. For each of the experiment a system configuration of 64 GB of RAM, 250 GB Hard Disk space, 8vCPU, in a AWS Sagemaker ML.R5.2xLarge instance is used. It takes almost 24 hours to finish one run of 500 generations with this configuration. For AE training dataset, Lenia class configuration is given in Table I where G in sub-script of Mu and Sigma are for Grown Function. Configuration for AE is provided in Table II and fixed configuration for Genetic Algorithm is provided in Table III.

Finally, all experiments with different combinations and parameters are displayed with their corresponding kernel out-

TABLE I
DATASET SPECIFICATIONS

Parameter	Value
Dataset Size	3000 frames
DPI	50
μ_G	0.31
σ_G	0.057
Dt	0.1
FPS	30
Kernel Size	16
Board Size	64

TABLE II
TRAINING CONFIGURATION FOR AE

Parameter	Value
Image Size	64x64
AE test size	0.30
AE Input Size	64x64
AE Hidden Size	36
AE Output Size	64x64
Epochs	300
Batch Size	128

puts from generation 1 and generation 500. In addition best fitness and average fitness are also plotted with mean-average performance plot within the average fitness plot in the Figure 10 for VoT, Figure 11 for AE and Figure 12 for AEVoT.

Once all the experiments mentioned in Figures 10, 11 and 12 are finished, we analysed and picked one experiment from AEVoT and another from VoT on the basis of performance and fitness growth. We also found that AE itself is not able to perform well as per the experiments shown in Figure 11. In our knowledge and inference we claim that AE itself could not find the complex behaviour because it just reconstructs and tries to measure MSE loss over the reconstruction which could not be that efficient as per variation over time used alone and with AE. In fact, AEVoT has best fitness performance when ran for 2500 generations once the configuration was handpicked from running 27 experiments (9 experiments from each of the fitness). We found, for AEVoT, Active cell threshold 0.5 and every 10^{th} frame is working well than other configurations as per fitness plot and kernel visualisation (with fitness value of 500.27). Moreover in VOT, Active cell threshold 0.1 for every 10^{th} frame is working well because of the same reason (with fitness value of 435.91). With the hope of getting higher fitness in future generations, it can clearly be seen in Figures 10 and 12 at configuration names 8P10_G500_M0.02_ALIVE_CELL_THRESHOLD

TABLE III
CONFIGURATION FOR GENETIC ALGORITHM

Parameter	Value
Kernel size	16
Board size	64
Mutation rate	0.02
Population size	10
Generation	500
Elites	1

0.1 EVERY_10TH_FRAMES and 7AE_VOT_P10_G500_M0.02_ALIVE_CELL_THRESHOLD0.5 EVERY_10TH_FRAMES_64x64size respectively. It is wise to infer that for variation over time, active cell threshold should be small and has to be calculated for a time delta of a significant value over span of total time steps. In our case we have a total time step of 100 where time delta is 10 and hence fitness is being calculated for every 10^{th} frame. Hence we provide a detailed analysis for these two experiments with kernel visuals from generation 1 to generation 2500 along with their long term best and average fitness plots in Figure 15. It is important to note that after longer generations more than 2500, the kernel centre of mass should have large pixel density of Lenia behaviour promoting pixel groups (can be seen in yellowish color map in the kernel plot(however all the dark pixels with dark blue color are outlined towards boundary in the plot. Moreover, one more claim can be made at this stage that if we let evolution to run for more than 2500 generations (in very very large generation size), we could also see an actual Lenia behaviour that we see with known kernel plots. Finally we plot five-fold-average for best and average fitness values of the same experiments shown in Figure 15 ran on five different run-times, which is shown in Figure 13

To compare our experiments with handcrafted kernels or known kernel, we perform six experiments keeping growth function and rest of the configuration same. Known kernels are very robust to perturbations. Even after doing mutations for these kernels they show similar behaviour and complexity measurement remains same which can be shown by the generation fitness curve as shown in Figure 16. At the first place, even after mutations there are very less changes happened in the fitness measurement value. In other words, we let evolution run for 500 generations and each generation has mutation impact of 2 pixels. However, the growth of fitness happens very slow in those generations, which means known kernels and their behaviour remains robust even after perturbations. For example, with the kernel configuration and best plots in the 3rd column shown in Figure 16 the growth has happened only at first few generations and then it becomes stable very soon. Rest of the time even after mutations the fitness measure remained same and hence the overall system remained robust in terms of pixel variation and pixel density. Finally, after handcrafting a kernel with a smoothed spherical Gaussian pixel distribution which is shown in Figure 14 along with the popular known growth function shows extra-ordinary complex behaviour as self-organisation. Metaphorically, it shows ring like bacteria pattern. All the results from such a kernel are shown in Figure 14. The configuration used here is same that is used in Figure 16

Full code with results is available on this link: bit.ly/evolenia
All experimental visualisations and animations were programmatically processed to produce a overall summary shown on our YouTube channel: bit.ly/leniaonyoutube

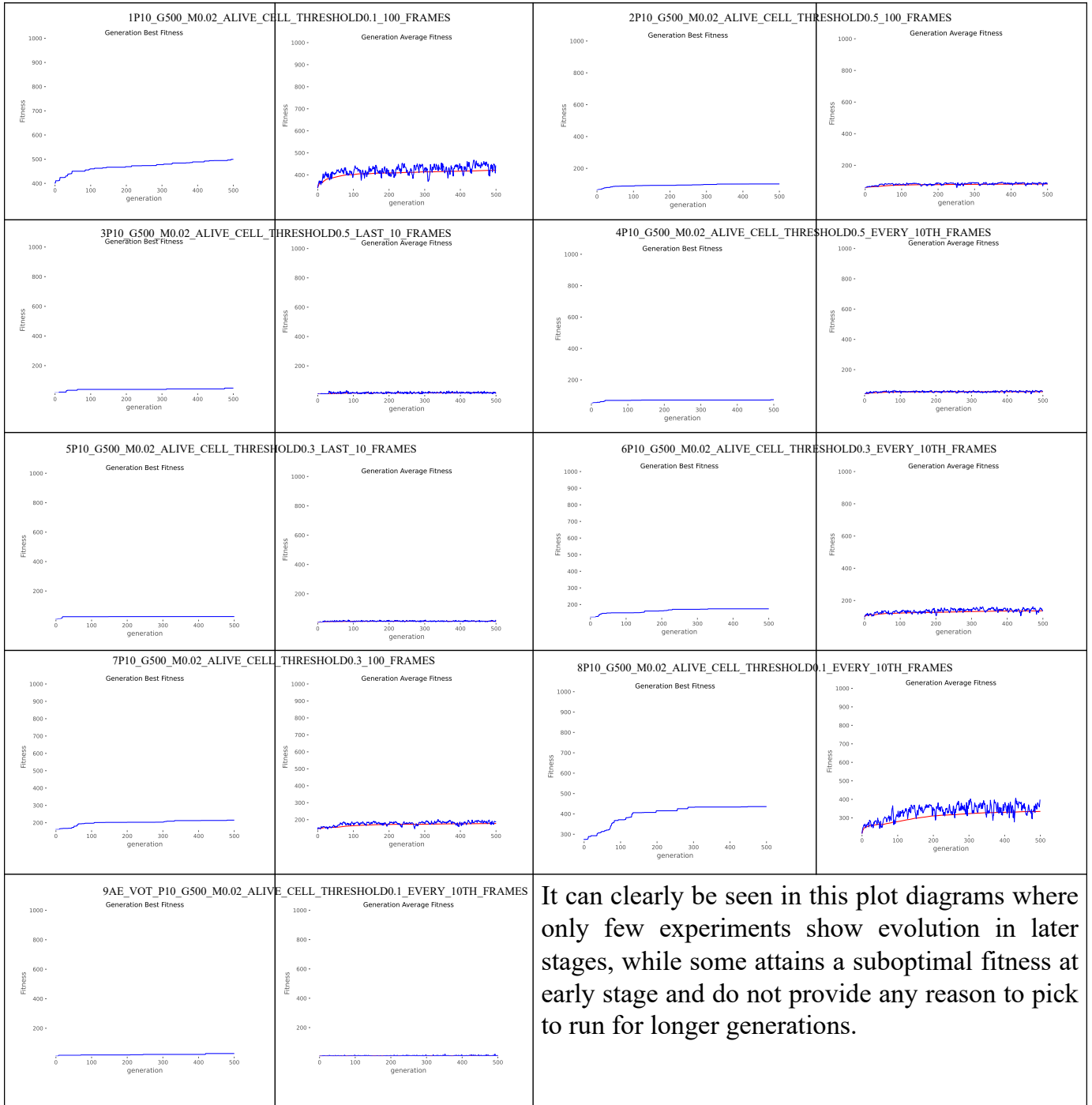


Fig. 10. Plot data for Variation over Time fitness. Configuration should be read as for example 1AE_36bottleneck_allframe Autoencoder fitness for bottleneck size of 36 ran for all 100 frames. Y-axis has upper limit of 1000.

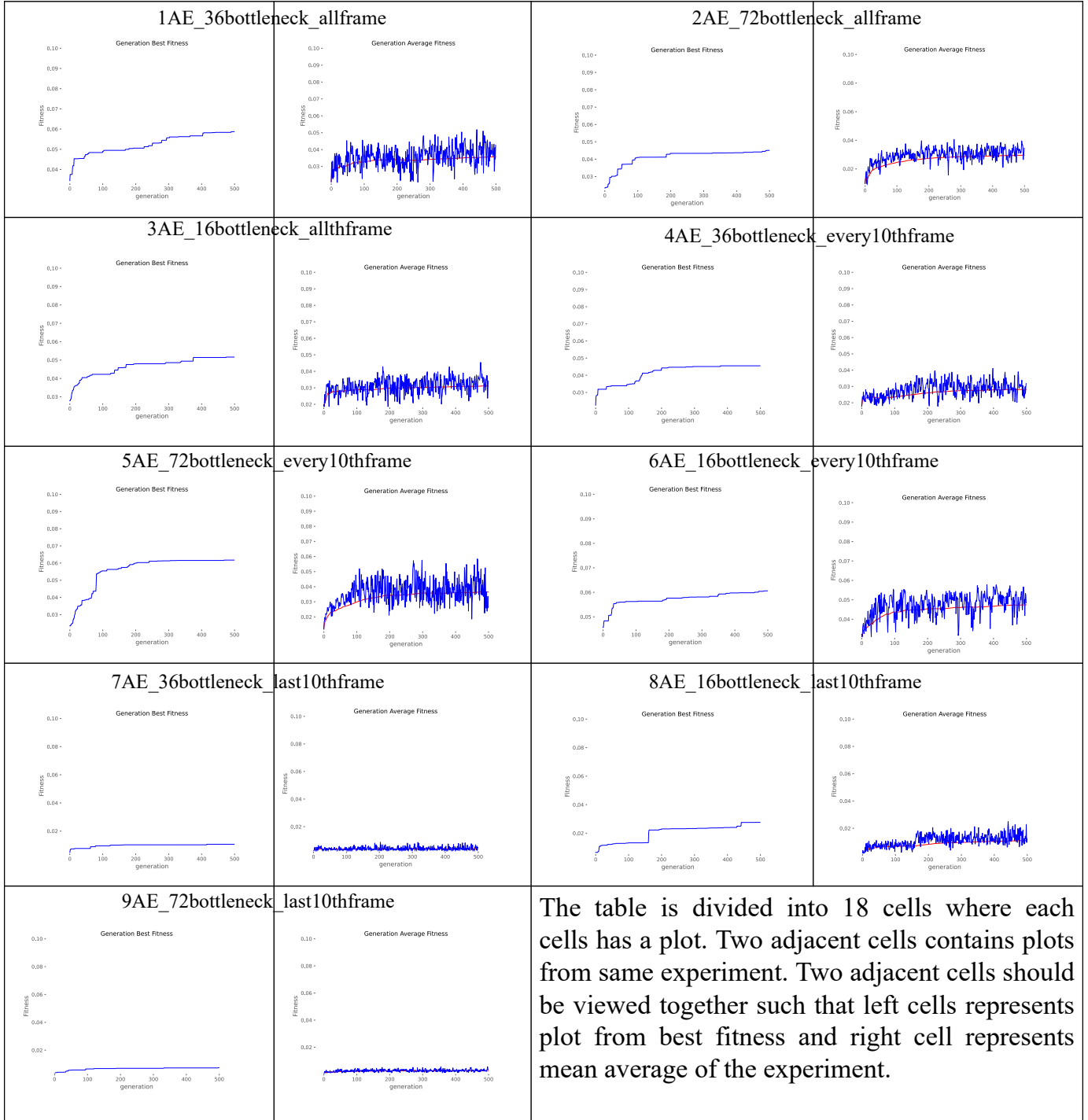


Fig. 11. Plot data for AutoEncoder fitness. Configuration should be read as for example 1P10_G500_M0.02_ALIVE_CELL_THRESHOLD 0.1_100_FRAMES should be read as population size (P) as 10 ran for 500 generations (G), with mutation rate (M) of 0.02, active cell threshold 0.1 for 100 frames. Y-axis has upper limit of 0.1.

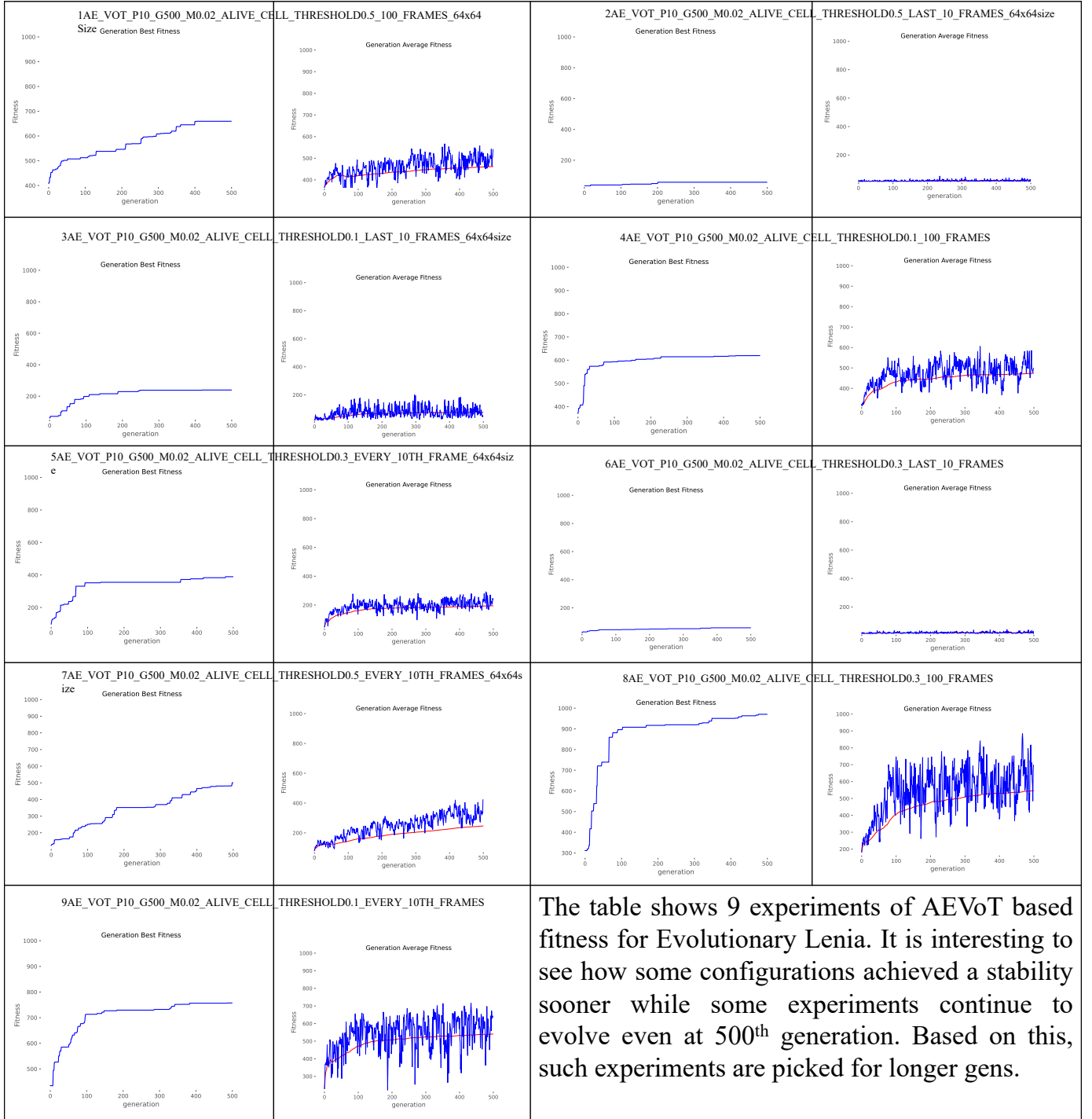


Fig. 12. Plot data for AutoEncoder with Variation over Time fitness. Configuration should be read as for example 1AE_VOT_P10_G500_M0.02_ALIVE_CELL_THRESHOLD0.5_100_FRAMES should be read as population size (P) as 10 ran for 500 generations (G), with mutation rate (M) of 0.02, active cell threshold 0.5 for 100 frames with 64 as input image size. Y-axis has upper limit of 1000.

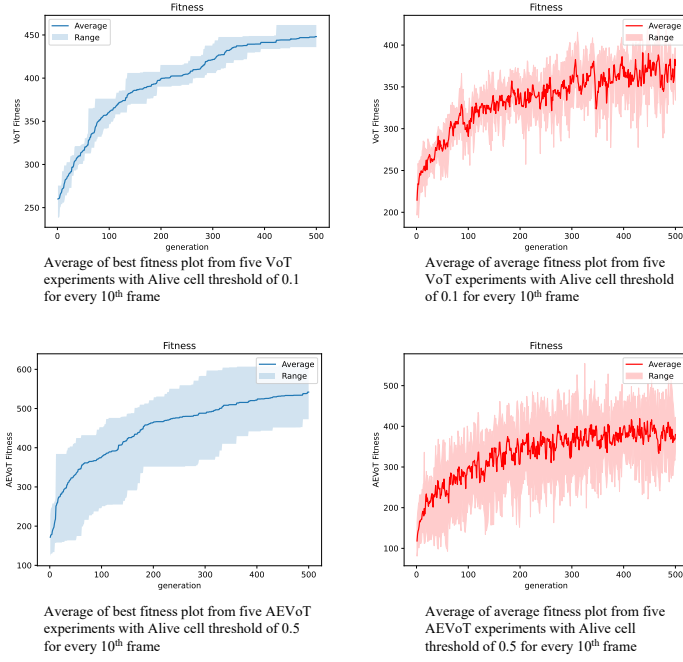


Fig. 13. Five-fold-averaging of best and average fitness values of five experiments with same configurations shown in Figure 15. First row shows VoT run over 500 generations (5 times) using Alive threshold of 0.1 and measuring over every tenth frame. Second row shows AEVoT run over 500 generations (5 times) using Alive threshold of 0.5 and measuring over every tenth frame.

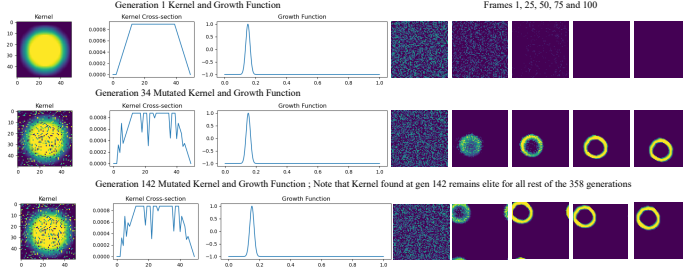


Fig. 14. Complex behaviour emerging in Evolutionary Lenia with the known kernel and its mutation for 500 generations

V. CONCLUSIONS

In this project, we started with understanding and learning Lenia which is a continuous CA. Lenia is an artificial life system that simulates complex emerging behaviour for a specific set of parameters in a large and wide existing parameter-space. To explore such wide space of parameters, we performed multiple experiments with the use of Genetic Algorithm. We exploited kernel information in order to make complex behaviour emerge within the Lenia system. In simpler words, we explored kernel space by mutating kernel pixels while starting with a randomly initialised kernel, keeping growth function fixed. Then, we measured complexity using three techniques AE, VoT and AEVoT that are proposed in this project and hence used these algorithms as fitness function for the Genetic Algorithm. Once we found a specific set of configuration performing well, we ran it for a little

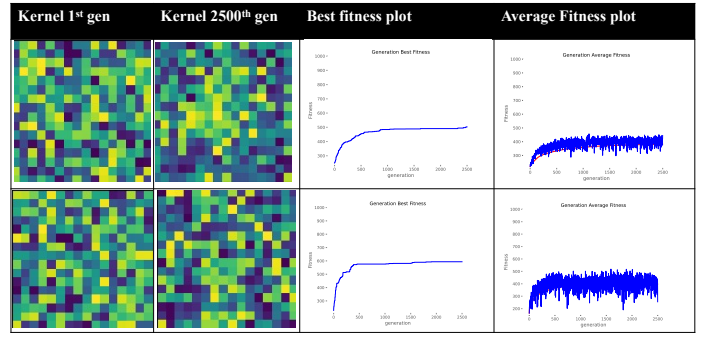


Fig. 15. Best and average fitness plots with kernel visualisations for the selected configuration running for 2500 generations. First row is VoT based simulation with configuration picked from Figure 10 8th row and second row is AEVoT based simulation with configuration picked from Figure 12 7th row

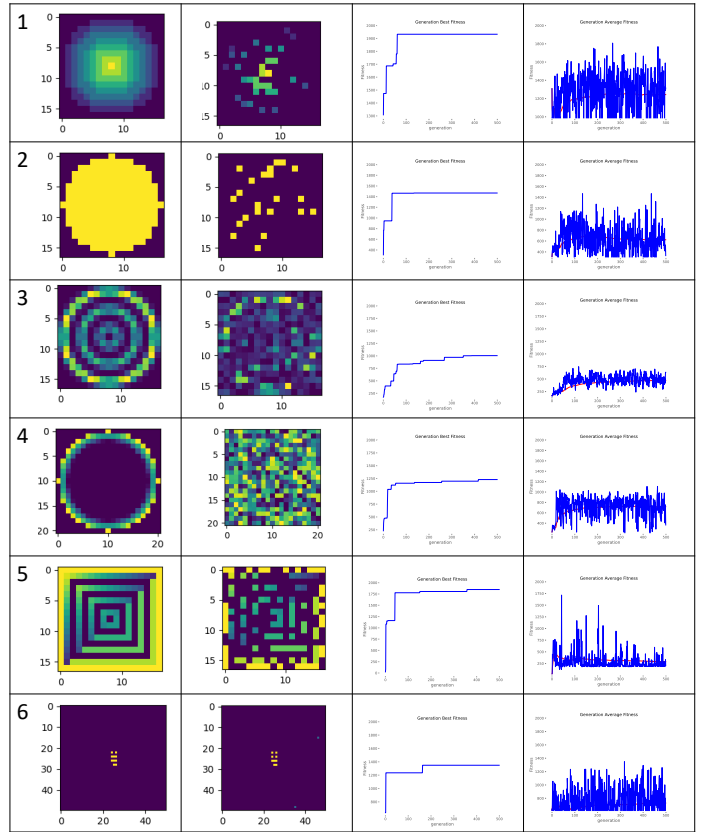


Fig. 16. Running 500 generation based simulation for known kernels. The configuration is kept same as the simulation shown in Figure 15 second row. First column shows the initial known kernel, second column shows mutated kernel after 500 generations. Additionally, third column shows best fitness plot and fourth column shows average fitness plot with cumulative mean-average line in between. (Plots have a y-limit of 2000)

longer generations and found a completely different behaviour emerging in Lenia system when compared with the initial random behaviour. Lenia, as a computational system, provides a platform for exploring emergent behaviors that mimic some of the properties of living systems, such as self-organization, pattern formation, and adaptation to changing environments. By studying these behaviors in Lenia, researchers can gain

insights into the underlying mechanisms that give rise to life-like systems. The GitHub repository for baselines and experimental code is github.com/s4nyam/APCSP

VI. ACKNOWLEDGEMENT

This project is part of Applied Computer Science Project supervised by Stefano Nichele at Østfold University College, Halden, Norway

REFERENCES

- [1] K. O. Stanley, J. Lehman, and L. Soros, “Open-endedness: The last grand challenge you’ve never heard of,” While open-endedness could be a force for discovering intelligence, it could also be a component of AI itself, 2017.
- [2] K. Gregor and F. Besse, “Self-organizing intelligent matter: A blueprint for an ai generating algorithm,” arXiv preprint arXiv:2101.07627, 2021.
- [3] B. W.-C. Chan, “Lenia-biology of artificial life,” arXiv preprint arXiv:1812.05433, 2018.
- [4] B. W.-C. Chan, “Lenia and expanded universe,” arXiv preprint arXiv:2005.03742, 2020.
- [5] A. Variengien, S. Nichele, T. Glover, and S. Pontes-Filho, “Towards self-organized control: Using neural cellular automata to robustly control a cart-pole agent,” arXiv preprint arXiv:2106.15240, 2021.
- [6] G. Hamon, M. Etcheverry, B. W.-C. Chan, C. Moulin-Frier, and P.-Y. Oudeyer, “Learning sensorimotor agency in cellular automata,” 2022.
- [7] E. Plantec, G. Hamon, M. Etcheverry, P.-Y. Oudeyer, C. Moulin-Frier, and B. W.-C. Chan, “Flow Lenia: Mass conservation for the study of virtual creatures in continuous cellular automata,” arXiv preprint arXiv:2212.07906, 2022.
- [8] Alexander Mordvintsev, Eyvind Niklasson, Ettore Randazzo, Google December 23, 2022 Particle Lenia and the energy-based formulation (no date). Available at: <https://google-research.github.io/self-organising-systems/particle-lenia/> (Accessed: March 6, 2023).
- [9] D. Medernach, T. Kowaliw, C. Ryan, and R. Doursat, “Long-term evolutionary dynamics in heterogeneous cellular automata,” in Proceedings of the 15th annual conference on Genetic and evolutionary computation, 2013, pp. 231–238.
- [10] H. Cisneros, J. Sivic, and T. Mikolov, “Evolving structures in complex systems,” in 2019 IEEE Symposium Series on Computational Intelligence (SSCI), 2019, pp. 230–237.
- [11] H. Cisneros, J. Sivic, and T. Mikolov, “Visualizing computation in large-scale cellular automata,” arXiv preprint arXiv:2104.01008, 2021.
- [12] Chan, B. W. C. (2023). Towards Large-Scale Simulations of Open-Ended Evolution in Continuous Cellular Automata. arXiv preprint arXiv:2304.05639.